

CV Final Project Report: Lane Boundary Detection from Highway Video

Team: Aria Askaryar, Souley Diallo, Varun Dhir

Abstract

This project implements a classical computer vision pipeline for lane boundary detection on forward-facing highway video. The goal was to detect the left and right lane boundaries, project a stable driving corridor back onto the original image, and measure how often the system succeeds under easy, moderate, and difficult road conditions. The implementation was based on the workflow in ``notebooks/proj_framework.ipynb`` and finalized in a runnable script, ``scripts/lane_detection_pipeline.py``, so the notebook logic could be executed consistently across all three source videos. The final system combines color thresholding, Gaussian smoothing, region-of-interest masking, perspective warping, RANSAC curve fitting, and Kalman-based temporal smoothing. Across 2,943 processed frames, the pipeline detected both lanes in 2,424 frames, for an overall two-lane detection rate of 82.4 percent. The easiest project clip achieved 100.0 percent detection, the challenge clip achieved 92.4 percent detection, and the harder challenge clip achieved 59.8 percent detection. These results show that a classical pipeline can work reliably on clearly marked highway footage, but it degrades when lane curvature, shadows, and weak paint increase ambiguity.

Problem Context

Lane boundary extraction is a practical problem in driver assistance and autonomous perception. Boeing is not building a highway autopilot in this class project, but the engineering problem is representative of a broader perception issue that matters in aerospace and industrial autonomy: a vision system must isolate a small set of safety-critical geometric features from noisy real-world imagery and do so consistently over time. In this project, the current state of the problem is that road scenes contain shadows, changing pavement texture, glare, faded paint, and curved lane geometry that make simple thresholding unstable. Deep learning solutions can solve more of these edge cases, but they require much more labeled data, training time, and compute. For a course project focused on core computer vision methods, a classical pipeline is a better fit because every stage is explainable and directly tied to topics covered in class.

Data and Inputs

The project used three public dashcam clips stored in ``data/videos/``:

- ``project_video.mp4``
- ``challenge_video.mp4``
- ``harder_challenge_video.mp4``

These videos span increasing difficulty. The project clip is mostly straight and well marked. The challenge clip includes more lighting variation and intermittent clutter. The harder challenge clip introduces stronger curvature, heavier shadows, and more unstable lane appearance. This progression is useful because it reveals not just whether the pipeline works, but also where and why it breaks. The final run also saved processed outputs to ``results/videos/`` and representative snapshots to ``results/snapshots/``.

Method

The notebook already contained the core idea for the solution, so the final implementation preserved those stages and made them reproducible in a standalone script. Each frame first passes through an HLS color transform so white and yellow lane markings can be isolated with threshold masks. The masked image is then blurred with a Gaussian filter to suppress small noise before downstream fitting. A trapezoidal region of interest is applied so the detector focuses on the roadway and ignores the sky, vehicles, trees, and roadside content that do not help lane estimation.

After preprocessing, the binary lane mask is warped into a bird's-eye view using a perspective transform. This step is important because the lane boundaries become closer to vertical and easier to fit as curves in the warped space. The warped image is divided into left and right halves. For each half, the nonzero lane pixels are collected and fit with a second-order polynomial of the form $x = Ay^2 + By + C$. Instead of using a simple least-squares fit, the project uses ``RANSACRegressor`` so outlier pixels from noise, pavement seams, or shadow edges do not dominate the model.

Frame-by-frame fits are still noisy, so temporal stabilization is added with a Kalman filter for each lane boundary. When the current frame produces a valid measurement, the tracker corrects its internal state; when the measurement is weak or missing, the tracker predicts a lane estimate from recent history. The two predicted curves are then validated as a pair using geometric constraints on lane width and lane ordering. If the pair is plausible, a polygon is drawn between the left and right curves, inverse-warped back into the camera view, and blended onto the original frame as a green lane overlay. If the pair fails validation, the frame is labeled as a fallback or failed detection frame.

Results

The completed pipeline was executed on all three source videos, and the metrics were written to ``results/lane_metrics.json`` and ``results/lane_metrics_summary.md``. The strongest result was on ``project_video.mp4``, where both lanes were detected in all 1,260 of 1,260 frames for a 100.0 percent success rate. The mean lane width in the warped fit was 697.19 pixels with a standard deviation of 17.01 pixels, which indicates very stable geometry in that clip. Runtime on this sequence was 20.09 processed frames per second, which is close to but slightly below the 25 FPS source rate.

The intermediate case, ``challenge_video.mp4``, produced 447 successful two-lane detections out of 484 frames, for a 92.4 percent success rate. The mean lane width was 588.54 pixels with a standard deviation of 77.15 pixels, which is noticeably less stable than the easy clip but still usable. This result shows that the combination of color masking, RANSAC fitting, and temporal smoothing remains

effective when there is moderate lighting variation and mild scene clutter. The processing speed on this clip was 35.03 frames per second, which exceeded the 29.97 FPS source rate.

The difficult sequence, `harder\_challenge\_video.mp4`, exposed the limits of the classical approach. The system detected both lanes in 717 of 1,199 frames, for a 59.8 percent success rate, and failed on 482 frames. The mean lane width dropped to 550.79 pixels with a standard deviation of 153.53 pixels, showing that the lane geometry was much less consistent frame to frame. Although left-lane and right-lane raw detections were still high individually, the pair often failed the geometric validation stage because one side drifted, curved away, or was corrupted by shadows and noisy candidate pixels. Processing speed also fell to 3.87 frames per second, which suggests that the difficult clip not only reduced accuracy but also increased fitting cost.

Across the full evaluation set, the pipeline processed 2,943 frames. Left-lane measurements were found in 98.7 percent of frames, right-lane measurements were found in 99.4 percent of frames, and both lanes were accepted together in 82.4 percent of frames. That gap between single-lane detection and valid paired-lane detection is important. It means the system often finds lane-like structure, but maintaining a stable and physically plausible lane corridor is the harder problem.

Discussion

The project met its main objective: it produced a working classical lane detector, generated lane-overlay videos, and provided frame-level metrics that explain when the approach succeeds and fails. The method is strongest on straight or gently curving roads with bright paint and consistent lighting. It degrades under the exact conditions expected from the proposal: faded lane markings, strong shadows, and curved lanes. The harder challenge clip demonstrates that a second-order fit in a fixed warped geometry is not always robust enough when the lane appearance changes rapidly across the scene.

There are several reasons for these failures. First, the HLS thresholds are hand tuned and therefore sensitive to illumination shifts. Second, the perspective transform is fixed rather than calibrated per scene, so it assumes a consistent camera pose and road geometry. Third, splitting the warped image into fixed left and right halves is simple, but it can mis-handle large curvature or temporary drift. Fourth, the Kalman filter stabilizes estimates only after a reasonable measurement already exists; it cannot recover cleanly from long stretches of poor evidence.

Recommended Next Steps

If the project were extended, the next improvements should be:

- Add adaptive thresholding or contrast normalization before color masking.
- Replace the fixed half-image split with histogram-based lane base initialization and sliding-window pixel aggregation.
- Use camera calibration and a better perspective model so curvature measurements are physically meaningful.

- Evaluate on manually labeled frames so error can be reported as lateral pixel deviation rather than only detection rate.
- Add confidence scoring and automatic reinitialization when the tracker drifts for several frames in a row.

Conclusion

This project demonstrates that a classical computer vision pipeline can solve lane boundary detection well enough to be persuasive on clean highway video and to illustrate key concepts from the course, including filtering, geometric transforms, robust fitting, and temporal estimation. The final implementation produced stable overlays and perfect performance on the easiest clip, strong performance on the moderate clip, and clearly documented failure behavior on the hardest clip. That outcome is useful because it shows both the strengths and the boundaries of the method. In other words, the project did not only generate positive results; it also generated the evidence needed to argue where a classical pipeline is sufficient and where more adaptive or learning-based methods would be required.